



SigmaStar Display

Application Development Reference Manual



© 2019 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.



{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

REVISION HISTORY

Revision No.	Description	Date
0.1	ÿ Initial release	04/06/2019



{SigmaStar Display}
{Smart Display}
{SSD201+1.0}

TABLE OF CONTENTS

REVISION HISTORYi

TABLE OF CONTENTS.....

ii 1. SDK Description 1

 1.1. Project directory description 1

 1.2. AppDemo directory introduction. 1

 1.2.1 How to compile Demo..... 1 1.2.2

 Parts implemented by DemoApp 2 1.3.

SSD201 Common Module Description..... 2

 1.3.1 SYS module..... 2 1.3.2 API

 usage of VDEC.. 3

 1.3.3 DISP 4

 1.3.4 Panel..... 4

 1.3.5 Framebuffer.....5



1. SDK description

1.1. Project directory description

```
beal.wu@sigmastar:~/master_i6/project$ ls
board  configs  image  kbuild  makefile  release  setup_config.sh  tools
```

board	Stores relevant configuration information, vdec firmware, mmap.ini, etc., such as the need to modify the framebuffer output format When you need to modify /project/board/i2m/SSC011A-S01A/config/fbdev.ini FB_HWWIN_FORMAT(5=ARGB8888, 6=ARGB1555) By default fb will be driven to the corresponding format
configs	Store and compile the sdk, generate the mirrored config, such as the first step of compilation ./setup_config.sh configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64
image	The main directory for compiling and packaging, including the compiled busybox image and source code, as well as the final compiled burner Record scripts and burn images.
kbuild	Kernel config and other settings needed to compile sdk
release	The header files, lib, and ko provided by the SDK, which contain the header files and data structures of each module
tools	some debug tools

1.2. AppDemo directory introduction

Get the source code of the demo and download it from github

```
beal.wu@sigmastar:~/master_i6/sdk/verify/application/smarttalk$ ls
3partyLib  demoApp  image.mk  README.txt  RunFile
```

3partyLib	third-party libraries and header files, some libs have not been used temporarily, mainly used minigui, xml, Cspotter Wait
demoApp	The main code logic code directory of the demoApp test program, which implements gui and media encapsulation
image.mk	When compiling the sdk, during the image creation process, copy the configuration files and media materials required for the demo to run
README.txt	manually executes the steps of the program, but spinor's /etc cannot be written, so it needs to be copied when making an image go in
RunFile	Configuration files and media assets needed to run the app

1.2.1 How to compile Demo

cd demoApp

./build_sample.sh (will build the demo according to the rules in CMakeList.txt)

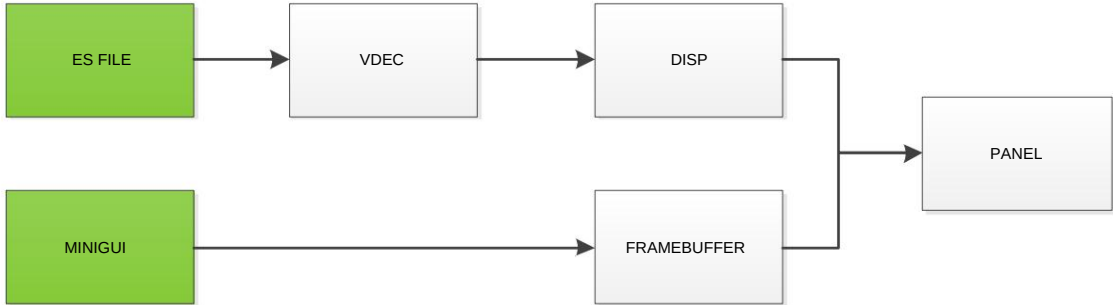
Successful compilation, the bin program of mginit will be generated in the build directory



{SigmaStar Display}
{Smart Display}
{SSD201+1.0}

1.2.2 The part implemented by DemoApp

Demo mainly implements UI and functional module display of decoding display. You can see this from demo



1.3. Description of common modules of SSD201

module	Function description
SYS	The SYS module is the memory management of the entire SDK, streaming control, frame rate setting, etc.
VDEC	Decoding, support H264, H265 decoding, maximum support 1080P 1. You need to set the width and height of the ES stream 2. It is necessary to set the width and height of the decoded output YUV. The output width and height must be the same as the input width and height of DISP. VDEC Supports reducing the width and height of ES streams, but does not support enlargement
DIVP	YUV image processing module, supports scaling and format conversion, often used to bind DISP and VENC at the same time
DISP	For the image display module, the Input port property needs to be set, and the width and height need to be consistent with the output width and height of the module bound to the previous level.
VENC	Encoding supports JPEG
For the configuration related to the PANEL display, refer to the demo to fill in the corresponding screen parameters	
GFX	2D engine, supports Fill, Blit, Blending, etc.
FB	Standard framebuffer interface, the device node is /dev/fb0, some extended ioctls are added, see mstarFb.h

The following introduces the main API usage of each common module in turn. You can refer to Demo to encapsulate and implement the customer's own media architecture.

1.3.1 SYS module

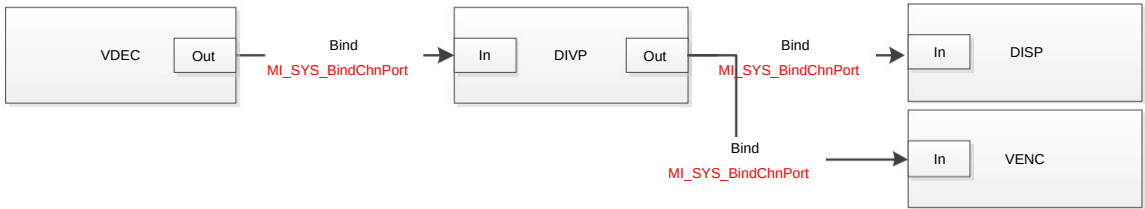
MI_S32 MI_SYS_Init(void); //This API needs to be called before using the module API

MI_S32 MI_SYS_Exit (void); //Corresponding to Init



If each module in the SDK needs to transmit YUV Stream, then it needs to bind the corresponding Port port, so that the upper-level module can handle it.

The YUV data will be sent to the next level module.



Note: Some modules only have Input Port, some modules only have Output Port, and some have Input/Output Port, as above



1.3.2 API usage of VDEC

```
MI_S32 ST_CreateVdecChannel(MI_S32 s32VdecChn, MI_S32 s32CodecType,
... MI_U32 u32Width, MI_U32 u32Height, MI_U32 u32OutWidth, MI_U32 u32OutHeight)
{
... MI_VDEC_ChnAttr_t stVdecChnAttr;
... MI_VDEC_OutputPortAttr_t stOutputPortAttr;

... memset(&stVdecChnAttr, 0, sizeof(MI_VDEC_ChnAttr_t));
... stVdecChnAttr.stVdecVideoAttr.u32RefFrameNum = 2;
... stVdecChnAttr.eVideoMode = E_MI_VDEC_VIDEO_MODE_FRAME;
... stVdecChnAttr.u32BufSize = 1 * 1024 * 1024;
... stVdecChnAttr.u32PicWidth = u32Width;
... stVdecChnAttr.u32PicHeight = u32Height;
... stVdecChnAttr.u32Priority = 0;
... stVdecChnAttr.eCodecType = s32CodecType;
... stVdecChnAttr.eInplaceMode = 0;

... STCHECKRESULT(MI_VDEC_CreateChn(s32VdecChn, &stVdecChnAttr));
... STCHECKRESULT(MI_VDEC_StartChn(s32VdecChn));
... if (u32OutWidth > u32Width)
... {
...     u32OutWidth = u32Width;
... }
... if (u32OutHeight > u32Height)
... {
...     u32OutHeight = u32Height;
... }
... stOutputPortAttr.u16Width = u32OutWidth;
... stOutputPortAttr.u16Height = u32OutHeight;
... MI_VDEC_SetOutputPortAttr(s32VdecChn, &stOutputPortAttr);

... return MI_SUCCESS;
} // end ST_CreateVdecChannel
```

ES流宽高
 ES流格式
 输出宽高，需小于等于ES流宽高

After creating the decoding channel, you need to call MI_VDEC_SendStream to send the network stream or video file to VDEC

```
if (MI_SUCCESS != (s32Ret = MI_VDEC_SendStream(vdecChn, &stVdecStream, s32TimeOutMs)))
{
... //ST_ERR("MI_VDEC_SendStream fail, chn:%d, 0x%X\n", vdecChn, s32Ret);
... fseek(readfp, -u32FpBackLen, SEEK_CUR);
}
```

Destroy the decoding channel:

```
MI_S32 ST_DestroyVdecChannel(MI_S32 s32VdecChn)
{
... MI_S32 s32Ret = MI_SUCCESS;

... s32Ret = MI_VDEC_StopChn(s32VdecChn); 先stop, 再destroy
... if (MI_SUCCESS != s32Ret)
... {
...     printf("%s %d, MI_VENC_StopRecvPic %d error, 0x%X\n", __func__, __LINE__, s32VdecChn, s32Ret);
... }
... s32Ret = MI_VDEC_DestroyChn(s32VdecChn);
... if (MI_SUCCESS != s32Ret)
... {
...     printf("%s %d, MI_VENC_StopRecvPic %d error, 0x%X\n", __func__, __LINE__, s32VdecChn, s32Ret);
... }

... return s32Ret;
}
```




1.3.3 DISP

Set the properties of disp dev and enable dev

```

- stPubAttr.stSyncInfo.u16Vact = stPanelParam.u16Height;
- stPubAttr.stSyncInfo.u16Vbb = stPanelParam.u16VSyncBackPorch;
- stPubAttr.stSyncInfo.u16Vfb = stPanelParam.u16VTotal - (stPanelParam.u16VSyncWidth + stPanelParam.u16Height +
- stPubAttr.stSyncInfo.u16Hact = stPanelParam.u16Width;
- stPubAttr.stSyncInfo.u16Hbb = stPanelParam.u16HSyncBackPorch;
- stPubAttr.stSyncInfo.u16Hfb = stPanelParam.u16HTotal - (stPanelParam.u16HSyncWidth + stPanelParam.u16Width +
- stPubAttr.stSyncInfo.u16Bvact = 0;
- stPubAttr.stSyncInfo.u16Bvbb = 0;
- stPubAttr.stSyncInfo.u16Bvfb = 0;
- stPubAttr.stSyncInfo.u16Hpw = stPanelParam.u16HSyncWidth;
- stPubAttr.stSyncInfo.u16Vpw = stPanelParam.u16VSyncWidth;
- stPubAttr.stSyncInfo.u32FrameRate = stPanelParam.u16DCLK * 1000000 / (stPanelParam.u16HTotal * stPanelParam.u16VTotal);
- stPubAttr.eIntfSync = E_MI_DISP_OUTPUT_USER;
- stPubAttr.eIntfType = E_MI_DISP_INTF_LCD;
- stPubAttr.u32BgColor = YUYV_BLACK;
MI_DISP_SetPubAttr(dispDev, &stPubAttr);
MI_DISP_Enable(dispDev);

```

屏参配置

使能disp device

Bind layer and disp dev

```

MI_DISP_BindVideoLayer(display, dispDev);
MI_DISP_EnableVideoLayer(display);

```

绑定layer

Set parameters and enable of disp input port

```

memset(&stInputPortAttr, 0, sizeof(stInputPortAttr));
u32InputPort = pstDispChnInfo->stInputPortAttr[i].u32Port;
ExecFunc(MI_DISP_GetInputPortAttr(DispLayer, u32InputPort, &stInputPortAttr, MI_SUCCESS));
stInputPortAttr.stDispWin.u16X = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16X;
stInputPortAttr.stDispWin.u16Y = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16Y;
stInputPortAttr.stDispWin.u16Width = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16Width;
stInputPortAttr.stDispWin.u16Height = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16Height;
stInputPortAttr.u16SrcWidth = pstDispChnInfo->stInputPortAttr[i].stAttr.u16SrcWidth;
stInputPortAttr.u16SrcHeight = pstDispChnInfo->stInputPortAttr[i].stAttr.u16SrcHeight;
ExecFunc(MI_DISP_SetInputPortAttr(DispLayer, u32InputPort, &stInputPortAttr, MI_SUCCESS));
ExecFunc(MI_DISP_GetInputPortAttr(DispLayer, u32InputPort, &stInputPortAttr, MI_SUCCESS));
ExecFunc(MI_DISP_EnableInputPort(DispLayer, u32InputPort, MI_SUCCESS));
ExecFunc(MI_DISP_SetInputPortSyncMode(DispLayer, u32InputPort, E_MI_DISP_SYNC_MODE_FREE_RUN, MI_SUCCESS));

```

设定DISP port的参数，可以理解为视频区域的设定，disp input port == disp video window

需和前级输出YUV宽高一致。

1.3.4 Panel

In the Panel part, the LCD of ttl only needs to fill in the screen parameters according to the spec, and then execute the following api. For the screen parameters, refer to the LCD header file of the demo

```

MI_PANEL_Init(E_MI_PNL_LINK_TTL);
MI_PANEL_SetPanelParam(&stPanelParam);

```

Panel初始化设定



{SigmaStar Display}

{Smart Display}

{SSD201+1.0}

1.3.5 Framebuffer

Initialize FB

```

g_fbFd = open(devfile, O_RDWR);
if (g_fbFd == -1)
{
    perror("Error: cannot open framebuffer device");
    exit(0);
}
//get fb fix screeninfo
if (ioctl(g_fbFd, FBIOGET_FSCREENINFO, &finfo) == -1)
{
    perror("Error: reading fixed information");
    exit(0);
}
//get fb var screeninfo
if (ioctl(g_fbFd, FBIOGET_VSCREENINFO, &vinfo) == -1)
{
    perror("Error: reading variable information");
    exit(0);
}
/* Figure out the size of the screen in bytes */
g_screensize = finfo.smem_len;

/* Map the device to memory */
frameBuffer =
    (char *) mmap(0, g_screensize, PROT_READ | PROT_WRITE, MAP_SHARED, g_fbFd, 0);
if (frameBuffer == MAP_FAILED)
{
    perror("Error: Failed to map framebuffer device to memory");
    exit(0);
}
// clear framebuffer
drawRect(0, 0, vinfo.xres, vinfo.yres, ARGB888_BLUE);

```

Framebuffer是标准的fb接口，如果是自行对接UI，那么需要如左侧初始化fb，然后直接对frameBuffer这个地址读写即可。fb会根据/config/fbdev.ini中的配置初始化设备

FB also extends some ioctl, such as colorkey and Hide/Show.

Show/Hide:

```

if (ioctl(g_fbFd, FBIOGET_SHOW, &bShown) < 0) {
    perror("Error: failed to FBIOGET_SHOW");
    exit(0);
}

```

ColorKey:

```

MI_FB_ColorKey_t colorKeyInfo;
if (ioctl(g_fbFd, FBIOGET_COLORKEY, &colorKeyInfo) < 0) {
    ST_ERR("Error: failed to FBIOGET_COLORKEY\n");
    exit(0);
}

colorKeyInfo.bKeyEnable = TRUE;
colorKeyInfo.u8Red = PIXEL888RED(u32ColorKeyVal);
colorKeyInfo.u8Green = PIXEL888GREEN(u32ColorKeyVal);
colorKeyInfo.u8Blue = PIXEL888BLUE(u32ColorKeyVal);

//convertColorKeyByFmt(&colorKeyInfo);
if (ioctl(g_fbFd, FBIOSET_COLORKEY, &colorKeyInfo) < 0) {
    ST_ERR("Error: failed to FBIOSET_COLORKEY");
    exit(0);
}

```

Extend ioctl:

```

#define FBIOGET_SCREEN_LOCATION _IOR(FB_IOC_MAGIC, 0x60, MI_FB_Rectangle_t)
#define FBIOSET_SCREEN_LOCATION _IOW(FB_IOC_MAGIC, 0x61, MI_FB_Rectangle_t)

#define FBIOGET_SHOW _IOR(FB_IOC_MAGIC, 0x62, MI_BOOL)
#define FBIOSET_SHOW _IOW(FB_IOC_MAGIC, 0x63, MI_BOOL)

#define FBIOGET_GLOBAL_ALPHA _IOR(FB_IOC_MAGIC, 0x64, MI_FB_GlobalAlpha_t)
#define FBIOSET_GLOBAL_ALPHA _IOW(FB_IOC_MAGIC, 0x65, MI_FB_GlobalAlpha_t)

#define FBIOGET_COLORKEY _IOR(FB_IOC_MAGIC, 0x66, MI_FB_ColorKey_t)
#define FBIOSET_COLORKEY _IOW(FB_IOC_MAGIC, 0x67, MI_FB_ColorKey_t)

#define FBIOGET_DISPLAYLAYER_ATTRIBUTES _IOR(FB_IOC_MAGIC, 0x68, MI_FB_DisplayLayerAttr_t)
#define FBIOSET_DISPLAYLAYER_ATTRIBUTES _IOW(FB_IOC_MAGIC, 0x69, MI_FB_DisplayLayerAttr_t)

#define FBIOGET_CURSOR_ATTRIBUTE _IOR(FB_IOC_MAGIC, 0x70, MI_FB_CursorAttr_t)
#define FBIOSET_CURSOR_ATTRIBUTE _IOW(FB_IOC_MAGIC, 0x71, MI_FB_CursorAttr_t)

```